

Firebase → Azure: Migrations-Playbook

Vollstaendige Handoff-Anleitung fuer einen anderen Chat (KI-Coding-Agent)

Dieses Dokument fasst alles zusammen, was bei der Umstellung der Next.js-App **AI Jobmap Moderator** von **Firebase (Auth + Firestore + Gemini + Pinecone)** auf **Microsoft Azure (NextAuth/Entra ID + Cosmos DB + Azure OpenAI + Key Vault + App Service)** gelernt wurde - als wiederverwendbares Rezept, damit ein anderer Chat eine vergleichbare App genauso migrieren kann. Schwerpunkt: die konkreten Schritte UND die hart erkaufte Stolperfallen (Abschnitt 11).

Aspekt	Wert
Quell-Stack	Next.js (App Router) + Firebase Auth + Firestore + @google/genai (Gemini) + Pinecone
Ziel-Stack	Next.js 14 + NextAuth v5 (Microsoft Entra ID) + Azure Cosmos DB + Azure OpenAI + Key Vault
Hosting	Azure App Service (Linux, Node 22, output: standalone, ZIP-Deploy)
Live-Ergebnis	https://jobmap-app.azurewebsites.net (vollstaendig migriert + verifiziert)
Zielgruppe	KI-Coding-Agent, der eine andere App 1:1 nach demselben Muster migriert

! Geheimnisse (Keys, Client-Secrets) sind in diesem Dokument bewusst NICHT enthalten. Sie werden zur Laufzeit aus Azure Key Vault bezogen bzw. per az-CLI gesetzt. Nicht-geheime Bezeichner (Subscription, Tenant, clientId, Endpunkte, Ressourcennamen) sind als konkrete Beispiele in Abschnitt 12 aufgefuehrt.

Inhalt

- 1 Zielarchitektur (Mapping Firebase → Azure)
- 2 Voraussetzungen & geteilte Azure-Basis
- 3 Phase 0 – Codebase-Inventur & Bereinigung
- 4 Phase 1 – Azure-Ressourcen provisionieren
- 5 Phase 2 – Auth-Migration (NextAuth + Entra ID)
- 6 Phase 3 – Daten-Migration (Firestore → Cosmos DB)
- 7 Phase 4 – KI-Migration (Gemini → Azure OpenAI) + Vektorsuche
- 8 Phase 5 – App-Settings & Secrets (Key Vault + Managed Identity)
- 9 Phase 6 – Build & Deploy (inkl. OneDrive-Falle)
- 10 Phase 7 – Verifikation
- 11 Hart erkaufte Lektionen / Gotchas (das Wichtigste)
- 12 Konkrete Werte dieser Migration (Beispiel)
- 13 Checkliste (Definition of Done)

1 Zielarchitektur (Mapping Firebase → Azure)

Schicht	Vorher (Firebase)	Nachher (Azure)
Auth (Client)	Firebase <code>useAuth()</code> / Google-Popup	NextAuth <code>useSession()</code> + <code>signIn("microsoft-entra-id")</code>
Auth (Server)	Firebase Admin <code>verifyIdToken</code>	<code>auth()</code> aus NextAuth (JWT-Session im HTTP-only-Cookie)
DB	Firestore (Subcollections)	Cosmos DB for NoSQL (flache Container)
LLM	Gemini (@google/genai)	Azure OpenAI (openai-SDK, AzureOpenAI-Client)
Embeddings	<code>text-embedding-004</code> (768-dim)	Azure <code>text-embedding-3-small</code> (1536-dim)
Vektorsuche	Pinecone (Namespace=projectId)	Cosmos DB Vektor-Index (VectorDistance, DiskANN)
Storage (Avatare)	Firebase Storage	Base64-Data-URL im Cosmos-Doc (kein Bucket)
Secrets	Service-Account-JSON / <code>.env</code>	Azure Key Vault + System-Managed-Identity
Hosting	Firebase Hosting / Vercel	Azure App Service (Linux, standalone ZIP)

Leitprinzip: Vertragstreue. Die Server-Auth-Helfer behalten dieselbe Signatur/Rueckgabeform wie vorher (`{ uid, email, profile }`), und der Client-`useAuth()`-Hook behaelt seinen Pfad und seine Felder. So bleibt die Umstellung der ~15 API-Routen und ~19 Client-Konsumenten minimal.

2 Voraussetzungen & geteilte Azure-Basis

- **az CLI** installiert und eingeloggt (`az login`); richtige Subscription gesetzt.
- **Geteilte Basis wiederverwenden:** Ein bestehender Resource-Group/Cosmos-Account/OpenAI-Ressource/App-Service-Plan kann von mehreren Apps geteilt werden. NEU pro App anlegen: eigene Cosmos-Datenbank + Container, ein Embedding-Deployment (falls noch nicht da), eine Entra-App-Registrierung, eine Web-App, ein Key Vault.
- **Node 22** lokal; npm.
- **Wichtig:** Build NICHT in einem OneDrive-synchronisierten Ordner ausfuehren (siehe Abschnitt 9 + 11).

3 Phase 0 – Codebase-Inventur & Bereinigung

- Firebase-Reste identifizieren, die ersetzt werden: `lib/firebase/*` (`admin`, `client`, `auth-context`, `storage`), der `Firebase-verify-auth`, `gemini.ts`, `pinecone.ts`, `Migrations-/Rollen-Skripte`, kaputte Auth-Form-Komponenten.
- Doppelte Admin-Seiten klaeren (alte vs. neue) - eine behalten, andere entfernen/redirecten.
- `next.config.mjs`: `output: "standalone"` ergaenzen (Pflicht fuer Azure-ZIP-Deploy).
- `package.json`: `Firebase/Gemini/Pinecone-Deps` raus; `@azure/cosmos`, `openai`, `next-auth` rein. Lockfile neu erzeugen, frisches `npm install`.
- Tipp: Den `Client-useAuth()`-Pfad (z.B. `@/lib/firebase/auth-context`) BEIBEHALTEN und nur die Implementierung tauschen - so muessen die ~19 Konsumenten nicht angefasst werden.

4 Phase 1 – Azure-Ressourcen provisionieren

4.1 Entra-App-Registrierung (OIDC)

! Die Audience MUSS AzureADandPersonalMicrosoftAccount sein, sonst koennen sich PERSOENLICHE Microsoft-Konten (z.B. @outlook.de) NICHT anmelden. Issuer = common.

```
az ad app create --display-name "MeineApp" \
  --sign-in-audience AzureADandPersonalMicrosoftAccount \
  --web-redirect-uris \
    https://MEINEAPP.azurewebsites.net/api/auth/callback/microsoft-entra-id \
    http://localhost:3000/api/auth/callback/microsoft-entra-id
# danach: Client-Secret erzeugen (az ad app credential reset) -> in Key Vault legen
```

4.2 Cosmos DB (Datenbank + Container + Vektor-Index)

- Account-Faehigkeit aktivieren: EnableNoSQLVectorSearch (einmalig pro Account).
- Datenbank anlegen (z.B. jobmap), dann FLACHE Container mit passenden Partitionsschluesseln.
- Der Karten-/Embedding-Container braucht eine **Vector Embedding Policy** (path /embedding, cosine, 1536) + DiskANN-Vektor-Index; das Embedding-Feld aus dem normalen Index ausschliessen.

```
// setup-cosmos.ts (Auszug) - Container-Definitionen
{ id: "users", pk: "/id" }
{ id: "projects", pk: "/id", ttl: true }
{ id: "cards", pk: "/projectId", vector: true, ttl: true }
{ id: "sessions|\"backlog|\"stakeholders|\"playbooks", pk: "/projectId", ttl: true }
{ id: "invitations", pk: "/id", ttlSeconds: 90d }
{ id: "auditLog", pk: "/uid", ttlSeconds: 2J }
// vector container:
vectorEmbeddingPolicy: { vectorEmbeddings:[{ path:"/embedding", dataType:"float32",
  distanceFunction:"cosine", dimensions:1536 } ] }
indexingPolicy: { excludedPaths:[{path:"/embedding/*"}], vectorIndexes:[{path:"/embedding",
  type:"diskANN"}] }
```

4.3 Azure OpenAI Deployments

! text-embedding-3-small ist in swedencentral nur als GlobalStandard-SKU verfuegbar (Standard nicht). Beim Deployment beruecksichtigen.

- Chat-Deployment (z.B. gpt-4.1-mini) und Embedding-Deployment (text-embedding-3-small, 1536-dim) anlegen.
- Endpoint + Key1 holen (Key -> Key Vault).

4.4 Web-App auf dem Plan

```
az webapp create -g <RG> -p <PLAN> -n MEINEAPP --runtime "NODE|22-lts"
az webapp config set -g <RG> -n MEINEAPP --startup-file "node server.js"
az webapp update -g <RG> -n MEINEAPP --https-only true
```

5 Phase 2 – Auth-Migration (NextAuth + Entra ID)

5.1 src/auth.ts (NextAuth v5)

- Provider MicrosoftEntraID, Issuer .../common/v2.0, scope openid profile email User.Read.
- Session-Strategie JWT (HTTP-only-Cookie). trustHost: true.
- jwt-Callback: beim Erst-Login das Cosmos-Profil lesen; falls keins vorhanden, NICHT automatisch ein autorisiertes Profil anlegen (siehe ORPHAN-Modell, Gotcha 5).
- avatarUrl aus dem JWT stripfen (Cookie-Groesse, Gotcha 6).
- trigger === "update" behandeln, um Profil/Rolle ohne Re-Login frisch aus Cosmos zu ziehen.

5.2 src/lib/server/verify-auth.ts (Durchsetzung)

Gleiche Signatur wie der alte Firebase-Helfer behalten: gibt { uid, email, profile } | NextResponse(401) zurueck. **Profil pro Anfrage FRISCH aus Cosmos laden** (nicht dem JWT vertrauen) - so wirken Rollenaenderungen sofort.

```
export async function verifyAuth(_req) {
  const session = await auth();
  if (!session?.user?.id) return NextResponse.json({error:"..."},{status:401});
  const fresh = await readItem<UserProfile>(usersContainer, uid, uid); // FRISCH
  return { uid, email: session.user.email ?? fresh.email, profile: fresh ?? session.user.p
  rofile };
}
// + requireRole(global) / requireProjectRole(project) fuer RBAC
```

5.3 Client useAuth()-Shim (Context, nicht nur Hook)

Als ECHTEN React-Context bauen (Provider laedt das frische Cosmos-Profil EINMAL via GET /api/users/profile und teilt es). Felder beibehalten: user, profile, loading, signIn/signOut, getIdToken, accountType, reloadProfile.

! Referenzielle Stabilitaet erzwingen: ALLE zurueckgegebenen Funktionen in useCallback wrappen, den Value memoisieren. Sonst -> Endlos-Refetch-Loop (Gotcha 3). getIdToken gibt bei Cookie-Auth null zurueck; der api-client nutzt das Cookie (kein Bearer).

6 Phase 3 – Daten-Migration (Firestore → Cosmos DB)

6.1 Zugriffsmuster (Firestore -> Cosmos)

Firestore	Cosmos
.doc(id).get()	container.item(id, pk).read()
Subcollection-Liste	SELECT * FROM c WHERE c.projectId=@pid ORDER BY c.createdAt DESC
.set(obj)	container.items.upsert({ id, ...obj })
.update(patch)	Read-Modify-Upsert (oder item.patch([...]) fuer Einzelfelder)
FieldValue.arrayUnion	Read-Modify-Upsert (Cosmos hat kein arrayUnion)
array-contains	ARRAY_CONTAINS(c.participants, @pfilter, true)

6.2 Generische Helfer (cosmos.ts)

```
readItem(container,id,pk) | queryItems(container,sql,params) |
upsertItem(container,item) | deleteItem(container,id,pk) | listByProject(container,pid)
```

6.3 Aufbewahrung per TTL mit Kaskade

! Cosmos hat KEINE kaskadierenden Loeschungen. Beim Archivieren muss die ttl auf JEDES Kind-Dokument in ALLEN Containern geschrieben werden - absturzsicher (per-Doc PATCH, Fehler isolieren, fehlgeschlagene IDs ins Audit-Log) plus ein idempotenter Reconcile-Endpoint. Sonst entstehen "Zombie"-Daten ohne Ablauf (DSGVO).

7 Phase 4 – KI-Migration (Gemini → Azure OpenAI) + Vektorsuche

Neuen Wrapper openai.ts mit gleichen Signaturen wie vorher: generateStructuredJson (chat.completions, response_format json_object, plus JSON-Reparatur), generateText, generateEmbedding(s). Prompts sind modell-agnostisch wiederverwendbar.

! HARTER Timeout am Client setzen (z.B. 45s) - der SDK-Default sind 10 Minuten; ein haengender Aufruf blockiert die single-threaded Node-Instanz.

```
const client = new AzureOpenAI({ endpoint, apiKey, apiVersion:"2024-10-21",
  timeout: 45000, maxRetries: 1 });
// Vektorsuche in Cosmos (ersetzt Pinecone):
SELECT TOP @k c.id, VectorDistance(c.embedding,@vec) AS score FROM c
WHERE c.projectId=@pid AND IS_DEFINED(c.embedding) ORDER BY VectorDistance(c.embedding,@ve
c)
```

Embedding direkt im Karten-Dokument speichern (embedding: number[]). Auf Lesepfaden den Vektor NICHT mitladen (Projektion ohne /embedding) - spart RU + Speicher.

8 Phase 5 – App-Settings & Secrets (Key Vault + Managed Identity)

- System-Managed-Identity auf der Web-App aktivieren; Key-Vault-Access-Policy (get/list) fuer diese Identity.
- Secrets (AUTH_SECRET, Entra-Client-Secret, Cosmos-Key, OpenAI-Key, ...) in den Key Vault legen.
- App-Settings als Key-Vault-Referenzen setzen:
@Microsoft.KeyVault(SecretUri=https://VAULT.vault.azure.net/secrets/NAME/).

Pflicht-App-Settings (Auszug):

Setting	Wert / Hinweis
AUTH_URL / NEXTAUTH_URL	exakt https://MEINEAPP.azurewebsites.net
AUTH_TRUST_HOST	true (zwingend hinter dem App-Service-Proxy)
AUTH_SECRET	Key-Vault-Referenz
AUTH_MICROSOFT_ENTRA_ID_ID / _SECRET	clientId / Key-Vault-Referenz
COSMOS_ENDPOINT / COSMOS_KEY / COSMOS_DATABASE	Endpoint / KV-Ref / DB-Name
AZURE_OPENAI_ENDPOINT / _API_KEY / _DEPLOYMENT / _EMBEDDING_DEPLOYMENT	Endpoint / KV-Ref / Deployment-Namen
WEBSITE_RUN_FROM_PACKAGE / ENABLE_ORYX_BUILD	1 / false (kein Server-seitiger Build)
PORT	8080

! Ohne AUTH_TRUST_HOST=true + exakte AUTH_URL -> NextAuth "UntrustedHost" bzw. redirect_uri-Mismatch.

9 Phase 6 – Build & Deploy (inkl. OneDrive-Falle)

! OneDrive-Falle (haeufigster Zeitfresser): next build bricht in einem OneDrive-Ordner bei "Collecting build traces" mit UNKNOWN copyfile errno -4094 ab (OneDrive-Filtertreiber). LOESUNG: Projekt in ein NICHT-OneDrive-Verzeichnis kopieren (robocopy), DORT bauen, dann zippen + deployen.

```
# 1) Quelle in Nicht-OneDrive-Dir spiegeln (node_modules/.next/.git ausschliessen)
robocopy <src>\src C:\build-dir\src /MIR
# 2) bauen
cd C:\build-dir && npm install && npm run build # output: standalone
# 3) standalone packen (static + public hineinkopieren)
copy .next\static -> .next\standalone\static ; copy public -> .next\standalone\public
Compress-Archive .next\standalone\* deploy.zip
# 4) deployen
az webapp deploy -g <RG> -n MEINEAPP --src-path deploy.zip --type zip
```

Diesen Ablauf als wiederverwendbares Skript (deploy.ps1) ablegen. Erfolg = RuntimeSuccessful.

10 Phase 7 – Verifikation

- Landing 200, NextAuth providers/csrf OK, signin 302 -> korrekte Microsoft-authorize-URL (client_id + redirect_uri + PKCE).
- Interaktiver MS-Login (auch persoenliches Konto), ein Projekt + Karte anlegen, einen KI-Call + semantische Suche.
- Geschuetzte API ohne Session -> 401. Key-Vault-Aufloesung pruefen (Export/Read funktioniert -> Secrets aufgeloeset).
- Log-Tail ohne Fehler (az webapp log tail).

11 Hart erkaufte Lektionen / Gotchas (das Wichtigste)

- 1. OneDrive-Build bricht ab.** next build im OneDrive-Ordner -> errno -4094. Immer ausserhalb bauen (Abschnitt 9).
- 2. PowerShell-CWD driftet.** Das Arbeitsverzeichnis driftet zwischen Aufrufen ins Eltern-Verzeichnis. Immer mit absolutem Pfad Set-Location arbeiten.
- 3. useAuth-Endlosschleife.** Ein Shim, der Funktionen pro Render neu erzeugt, loest in den ~19 Konsumenten (createApiClient(getIdToken) + fetch-in-effect) eine Endlos-Refetch-Schleife aus. FIX: alle Funktionen useCallback, Value useMemo, instabile Hook-Werte (z.B. next-auth update) per Ref kapseln.
- 4. Stale-Client-Rolle.** Das JWT bettet das Profil beim Login ein; Rollenaenderungen in Cosmos schlagen client-seitig erst beim Re-Login durch. FIX server-seitig: verifyAuth laedt das Profil PRO ANFRAGE frisch. FIX client-seitig (Anzeige): frisches Profil via GET /api/users/profile holen, nicht dem JWT vertrauen.
- 5. Selbst-Provisionierung (ORPHAN).** Issuer "common" erlaubt JEDEM Microsoft-Konto den Login. Beim Erst-Login NICHT automatisch ein autorisiertes Profil anlegen. Stattdessen Rolle ORPHAN ohne Cosmos-Profil; echtes Profil entsteht NUR beim Einloesen einer E-Mail-gebundenen Einladung (Accept-Endpoint prueft invitation.email === auth.email).
- 6. Avatar-Data-URL im JWT.** Ein ~30KB Foto-Data-URL sprengt das Session-Cookie-Limit und ein Reload liest das JWT ohnehin nicht neu aus Cosmos. FIX: avatarUrl aus dem JWT stripfen; Bild via GET /api/users/profile lesen; nach dem Speichern reloadProfile() statt window.location.reload().
- 7. Cosmos: keine Kaskaden-Loeschung.** Aufbewahrung/Loeschung ueber Kind-Container manuell kaskadieren - absturzsicher (per-Doc, Fehler isolieren, IDs protokollieren) + idempotenter Reconcile-Endpoint.
- 8. Cosmos: Partition-Key nicht per upsert aenderbar.** Wer ein Feld aendert, das der Partition-Key ist (z.B. Audit /uid beim Anonymisieren), erzeugt mit upsert ein ZWEITES Doc und laesst das Original stehen. FIX: Original loeschen + unter neuem PK neu einfuegen - und zwar insert-DANN-delete (Crash-Sicherheit).
- 9. Cosmos: keine atomaren Cross-Container-Transaktionen / kein arrayUnion.** Read-Modify-Write verwenden (z.B. Teilnehmerlisten). Bei Bedarf optimistische Nebenlaeufigkeit (etag) - bei geringer Last aber oft "premature complexity".
- 10. Entra-Audience.** MUSS AzureADandPersonalMicrosoftAccount sein, sonst koennen sich persoenliche Outlook-Konten nicht anmelden.
- 11. AUTH_TRUST_HOST + AUTH_URL.** Hinter dem App-Service-Proxy zwingend true + exakte URL, sonst UntrustedHost / redirect_uri-Mismatch.
- 12. output: standalone + static/public kopieren.** Pflicht fuers ZIP-Deploy; static + public muessen ins standalone-Verzeichnis kopiert werden, sonst fehlen Assets.
- 13. Embedding-SKU.** text-embedding-3-small in swedencentral nur als GlobalStandard-SKU.

14. Vektorsuche-Capability. Account-Capability EnableNoSQLVectorSearch + Vector-Policy + DiskANN-Index noetig; sonst schlaegt VectorDistance fehl.

15. Bearer-Leftovers. Nach Umstieg auf Cookie-Auth gibt getIdToken null zurueck. Achtung auf Seiten, die noch auf einen Bearer-Token gaten (z.B. eine Invite-Accept-Seite) - die liefern dann NIE. Auf Cookie-Fetch umstellen.

16. OpenAI-Timeout. SDK-Default 10min -> harter 45s-Timeout am Client (single-threaded Instanz).

17. i18n __session-Cookie. Firebase-Hosting leitete nur __session weiter; auf Azure ist dieser Cookie obsolet UND schaedlich (ueberschrieb NEXT_LOCALE). Nur noch NEXT_LOCALE lesen/schreiben.

18. Zod nach Auth. Eingabevalidierung NACH der Auth ausfuehren (401 vor 400). Array-Felder validieren - sonst kann ein .join() auf einem Nicht-Array die Instanz mit 500 crashen.

19. Theme/Light-Dark (optional). Bei next-themes: , darkMode:"class", semantische Tokens (.dark-Block); hartkodierte Neutralfarben -> Tokens, sonst bricht ein Modus.

12 Konkrete Werte dieser Migration (Beispiel, nicht geheim)

Diese App teilte sich eine bestehende "PulseCraft AI"-Azure-Basis. Werte als Vorlage - fuer eine andere App entsprechend ersetzen. (KEINE Secrets.)

Ressource	Wert
Resource Group	pulsecraft-prod-rg
Region	West Europe (OpenAI-Region swedencentral)
Cosmos-Account / DB	pulsecraft-prod-cosmos / Datenbank "jobmap"
OpenAI-Ressource	pulsecraft-prod-openai (Endpoint pulsecraft-prod-openai-20f84.openai.azure.com)
Chat- / Embedding-Deployment	gpt-41-mini-prod / text-embedding-3-small (1536)
App-Service-Plan	pulsecraft-prod-plan (B2 Linux, Node 22)
Web-App	jobmap-app -> https://jobmap-app.azurewebsites.net
Key Vault	jobmap-kv-20f84
Entra-App (clientId)	1e3e044f-1071-4959-9df5-271d952a0ecd (Audience AzureADandPersonalMicrosoftAccount)

Subscription/Tenant sind nicht-geheime Bezeichner und koennen bei Bedarf ergaenzt werden; alle KEYS und CLIENT-SECRETS liegen ausschliesslich im Key Vault.

13 Checkliste (Definition of Done)

- Entra-App (richtige Audience) + Redirect-URIs (prod + localhost) angelegt; Client-Secret im Key Vault.
- Cosmos-DB + Container + Vektor-Index; Vektorsuche-Capability aktiv.
- Chat- + Embedding-Deployment vorhanden.
- auth.ts + verify-auth.ts (frisches Profil) + useAuth-Context (frisch via GET) umgestellt.
- Alle API-Routen auf NextAuth + Cosmos + Azure OpenAI migriert; Bearer-Leftovers entfernt.
- Managed Identity + Key-Vault-Referenzen; alle Pflicht-App-Settings gesetzt (AUTH_TRUST_HOST, AUTH_URL, ...).
- Build AUSSERHALB OneDrive; standalone gepackt; az webapp deploy = RuntimeSuccessful.

- Verifiziert: MS-Login (auch persoenliches Konto), CRUD, KI-Call, Vektorsuche, 401 ohne Session, Key-Vault-Aufloesung, Log-Tail sauber.

Ende des Playbooks. Erstellt aus der real durchgefuehrten + live verifizierten Migration der App AI Jobmap Moderator (Firebase -> Azure).